

CV Project Report – Oxford102 Flower Dataset

Elective seminar: Artificial intelligence

17.4.2023

Risyad, Muhammad Emir

Dedeakayoğulları, Selman

Remmler, Mari Susanna

Introduction

In this project, we try to classify the OXFORD102 Flowers Dataset using several deep learning models. We explored various models, from a custom model to pretrain models. Our goal was to evaluate the performance of these models and compare their results.

Oxford102 Flower Dataset

Oxford 102 Flower is an image classification dataset consisting of 102 flower categories, each image is labeled with the name of the flower species it belongs to. The flowers chosen to be flower commonly occurring in the United Kingdom. Each class consists of between 40 and 258 images. The images have large scale, pose and light variations. In addition, there are categories that have large variations within the category and several very similar categories.

	Aphine sea holly	43		Buttercup	71		Fire lily	40
	Anthurium	105		Californian poppy	102		Foxglove	162
	Artichoke	78		Camelia	91		Frangipani	166
	Azalea	96		Canna lily	82		Fritillary	91
	Ball moss	46		Canterbury bells	40		Garden phlox	45
	Balloon flower	49		Cape flower	108		Gaura	67
	Barbeton daisy	127		Carnation	52		Gazania	78
	Bearded iris	54		Cautleya spicata	50		Geranium	114

Figure 1 OXFORD102 flowers dataset

Custom Model

Making a model from scratch works sometimes for basic problems. But those models are generally not capable to get some good results for more complex problems. Even if you try implement a larger model, it is going to be harder to train. You need more epochs and more computational power.

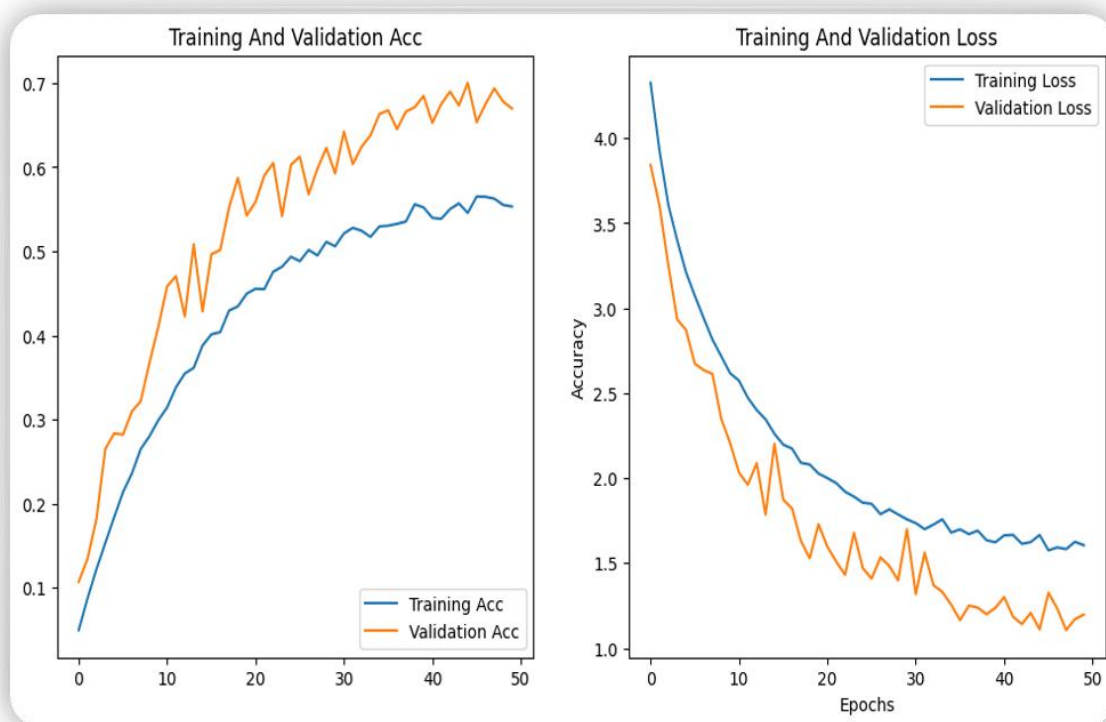


Figure 2 Custom model training history

Our test led to accuracy of approximately 70% after 50 Epochs.

RESTNET50

RestNet50 is a convolutional neural network architecture that is often used for image classification tasks. It consists of 50 layers, 48 convolutional layers, 1 MaxPool layer and 1 average pool layer. The architecture was introduced in the 2015 paper "Deep Residual Learning for Image Recognition" by Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. One of the key innovations of the ResNet architecture is the use of residual blocks, which allow the network to learn more complex features without suffering from the vanishing gradient problem.

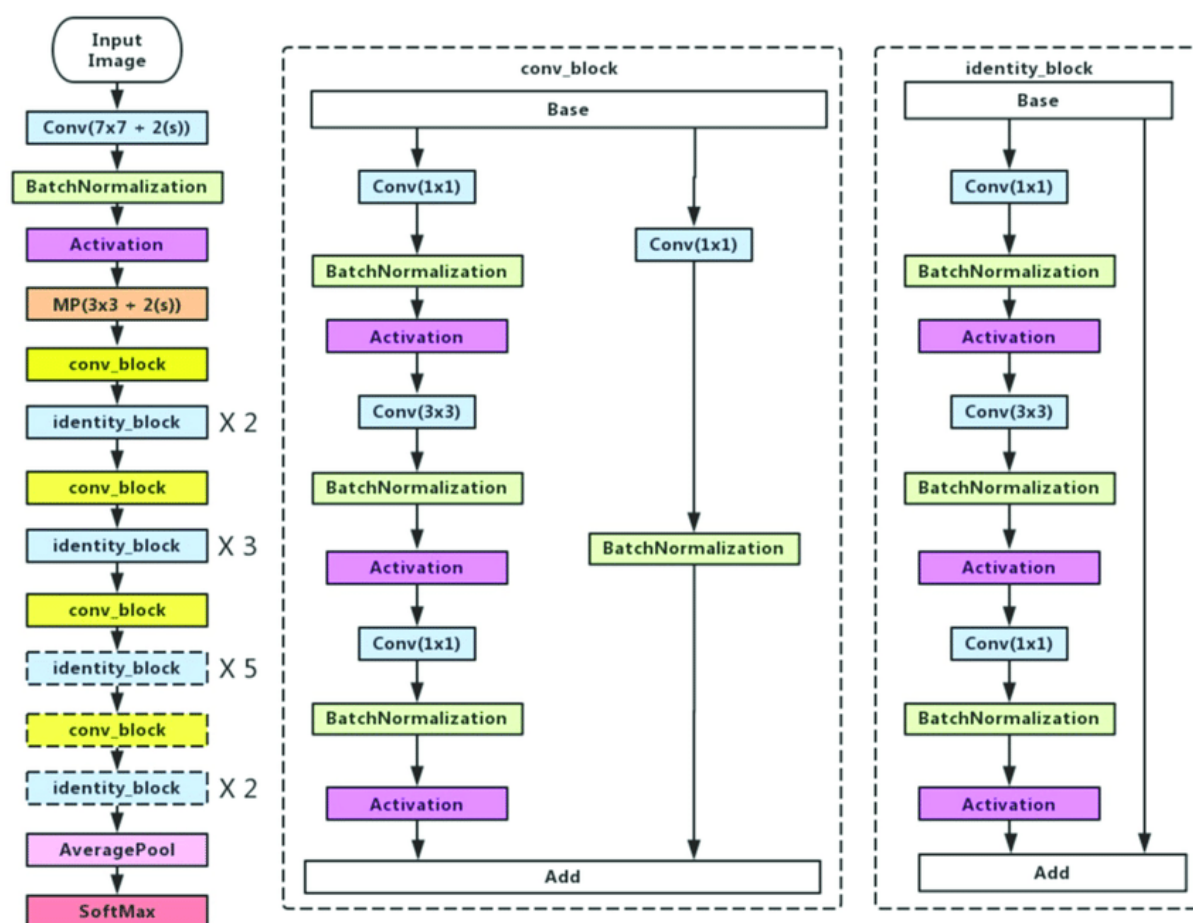


Figure 3 RESTNET50 Architecture

We tried to do fine tuning with the model to fit better with our dataset, because this model can easily become overfitted if not controlled.

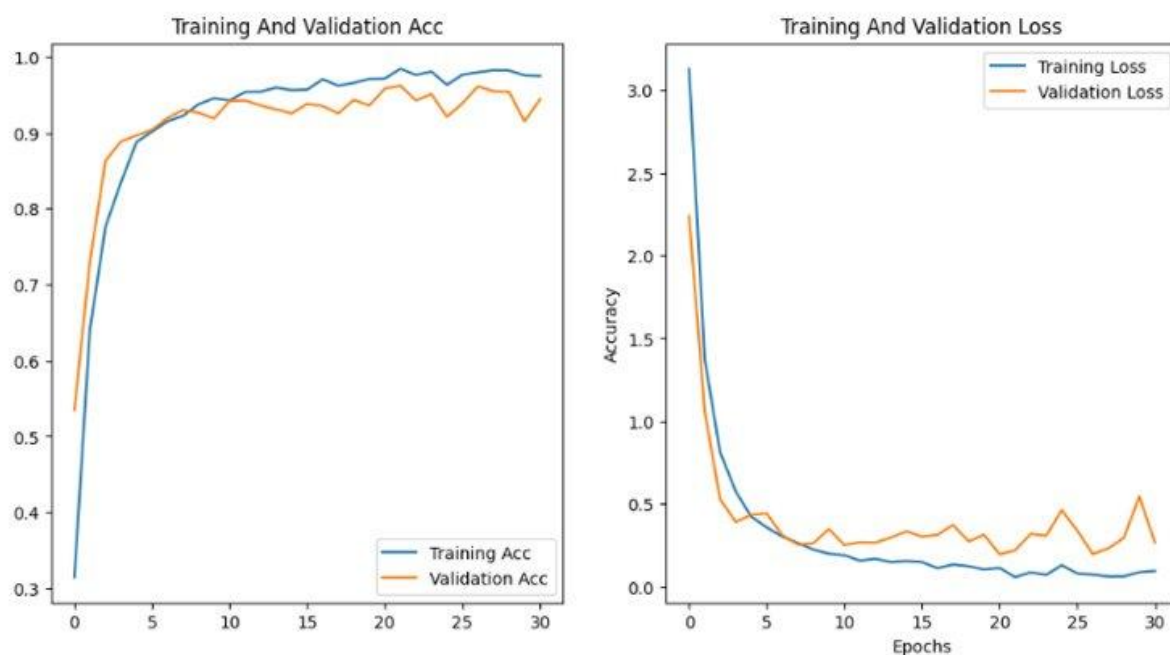


Figure 4 RESTNET50 training history

We manage to get 95% accuracy from this model.

VGG16

VGG16 is a convolutional neural network architecture that was introduced in 2014. It is named "VGG" because it was developed by the Visual Geometry Group at the University of Oxford. VGG16 consists of 16 layers, including 13 convolutional layers and 3 fully connected layers. It is used for image classification and recognition tasks, and has achieved state-of-the-art performance on the ImageNet dataset. The architecture is characterized by its simplicity and uniformity, with all convolutional layers having a 3x3 filter size and all pooling layers having a 2x2 filter size.

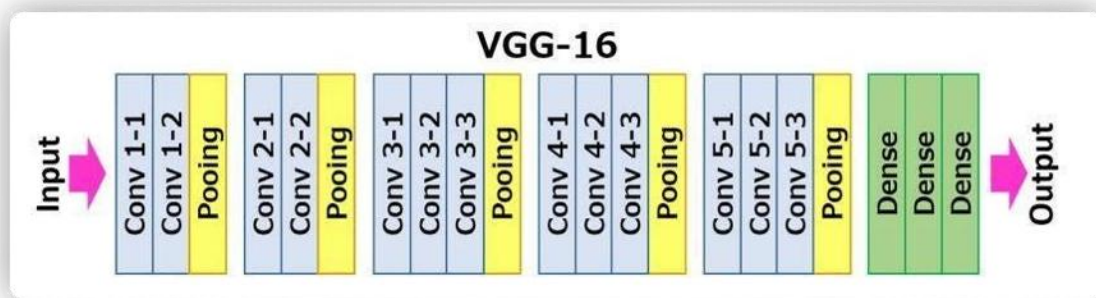


Figure 5 VGG16 architecture

```
from keras.applications.vgg16 import preprocess_input

data_train = DataGenerator("train", import_labels("train_labels.csv"), 16,
                           (224, 224), preprocess_input, use_augmentation=True)

data_test = DataGenerator("test", import_labels("test_labels.csv"), 16,
                           (224, 224), preprocess_input)

vgg_base = tf.keras.applications.VGG16(include_top=False, input_shape=(224, 224, 3))

vgg_model = tf.keras.models.Sequential([vgg_base, tf.keras.layers.GlobalAveragePooling2D(),
                                         tf.keras.layers.Dense(512, activation="relu"),
                                         tf.keras.layers.Dropout(0.5),
                                         tf.keras.layers.Dense(102, activation="softmax", name="output_layer")])

vgg_model.compile(loss="categorical_crossentropy",
                  optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
                  metrics=["accuracy"])

results = vgg_model.fit(data_train, epochs=50, validation_data = data_test, callbacks=[tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=5),
                                                                                   tf.keras.callbacks.ModelCheckpoint("model.h5",
                                                                                   monitor='val_accuracy', mode='max', save_best_only=True)])
```

Figure 6 VGG16 code

Unfortunately, we could not use VGG16 due Resource Exhaustion Error.

```
Epoch 1/50
3/342 [.....] - ETA: 2:02 - loss: 41.7337 - accuracy: 0.0417

Output exceeds the size limit. Open the full output data in a text editor
ResourceExhaustedError: Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_25012\567890317.py in
17 optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
18 metrics=["accuracy"])
--> 19 results = vgg_model.fit(data_train, epochs=50, validation_data = data_test, callbacks=[tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=5),
20                                                                                   tf.keras.callbacks.ModelCheckpoint("model.h5",
21                                                                                   monitor='val_accuracy', mode='max', save_best_only=True)])

c:\Users\emirr\anaconda3\lib\site-packages\keras\utils\traceback_utils.py in error_handler(*args, **kwargs)
65 except Exception as e: # pylint: disable=broad-exception
66     filtered_tb = _process_traceback_frames(e._traceback_)
--> 67     raise e.with_traceback(filtered_tb) from None
68 finally:
69     del filtered_tb

c:\Users\emirr\anaconda3\lib\site-packages\tensorflow\python\tracing\execute.py in quick_execute(op_name, num_outputs, inputs, attrs, ctx, name)
52 try:
53     ctx.ensure_initialized()
--> 54     tensors = pywrap_tfex.Py_Execute(ctx._handle, device_name, op_name,
55                                     inputs, attrs, num_outputs)
56 except core._NotOkStatusException as e:

ResourceExhaustedError: Graph execution error:
...
Node: 'gradient_tape/sequential_3/vgg16/block1_conv2/Conv2D/Conv2DBackpropInput'
OOM when allocating tensor with shape [16, 64, 224, 224] and type float on /job:localhost/replica:0/task:0/device:GPU:0 by allocator GPU_0_bfc
[[[tf.node.gradient_tape/sequential_3/vgg16/block1_conv2/Conv2D/Conv2DBackpropInput]]]]
Hint: If you want to see a list of allocated tensors when OOM happens, add report_tensor_allocations_upon_oom to RunOptions for current allocation info. This isn't available when running in Eager mode.
[Op:__inference_train_function_70841]
```

Figure 7 VGG16 error message

MobileNetV2

MobileNetV2 is a convolutional neural network architecture that was introduced in 2018. It is designed for mobile and embedded vision applications, where computational resources are limited. MobileNetV2 is based on the MobileNet architecture, but with significant improvements in terms of accuracy and speed. It uses a technique called "inverted residual blocks", which consist of a depthwise convolution (which applies a single filter per input channel) followed by a pointwise convolution (which applies a linear combination of filters to the outputs of the depthwise convolution). This allows for a significant reduction in computational cost, while still maintaining high accuracy.

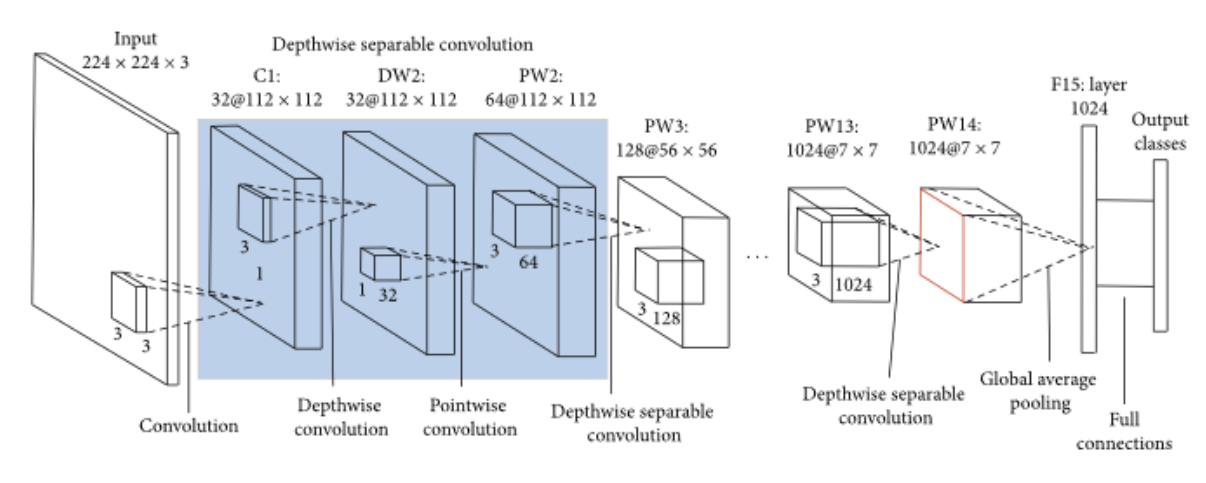


Figure 8 MobileNetV2 architecture

We got the highest accuracy from this model, which is 95.6% accuracy.